

คำตอบคำถามวิเคราะห์ (Case Study Analysis)

กรณีศึกษาที่ 3: ระบบคำนวณยอดขายรวม (Sales Summary System)

ข้อ 1 คำสั่งหลักใดทำงาน (n) ครั้ง

คำสั่งหลักที่ทำงาน n ครั้ง คือ คำสั่งภายใน **Loop (for-loop)** ที่วนอ่านข้อมูลยอดขายในอาร์เรย์ *sales* ทีละตัว โดยแต่ละรอบของลูปประกอบด้วยคำสั่งหลัก ดังนี้:

- คำสั่งบวกสะสมยอดขายรวม: $total += sale$; (ทำงาน n ครั้ง)
- คำสั่งเปรียบเทียบค่าสูงสุด: $if (sale > maximum)$ (ทำงาน n ครั้ง)

รายการคำสั่ง / ขั้นตอน	จำนวนครั้งที่ทำงาน (กรณีทั่วไป n ครั้ง)	จำนวนครั้งเมื่อ $n = 5$ (ข้อมูลตัวอย่าง)
กำหนดค่าเริ่มต้นตัวแปร ($total = 0, maximum = sales[0]$)	2 ครั้ง (ค่าคงที่)	2 ครั้ง
คำสั่งใน Loop หลัก ($total += sale$ & เปรียบเทียบ $maximum$)	n ครั้ง	5 ครั้ง
การปรับอัปเดตค่า $maximum = sale$	1 ถึง n ครั้ง (ขึ้นกับข้อมูล)	2 ครั้ง (ค่า 1575.25, 2300.00)
คำนวณค่าเฉลี่ย (<i>average</i>) และ คืนค่า (<i>return</i>)	2 ครั้ง (ค่าคงที่)	2 ครั้ง

สรุป: สำหรับข้อมูลในโจทย์ที่มี $n = 5$ ลูปหลักจึงทำงานทั้งหมด **5 ครั้ง**

ข้อ 2 Time Complexity ของอัลกอริทึมเป็นเท่าใด

Big O = $O(n)$

Big Ω = $\Omega(n)$

Big Θ = $\Theta(n)$

คำอธิบาย: อัลกอริทึมนี้วนลูปผ่านข้อมูลยอดขายในอาร์เรย์เพียง 1 รอบ (Single Pass) โดยในแต่ละรอบของการวนลูปจะทำการคำนวณผลรวมสะสมและเปรียบเทียบหาค่าสูงสุด ซึ่งใช้เวลาประมวลผลเป็นค่าคงที่ จากนั้นคำนวณค่าเฉลี่ยภายนอกลูปเพียง 1 ครั้ง ส่งผลให้เวลาประมวลผลเพิ่มขึ้นแบบเชิงเส้น (Linear Time) ตามจำนวนข้อมูล n

ข้อ 3 Auxiliary Space Complexity เป็นเท่าใด

Auxiliary Space Complexity = $O(1)$

คำอธิบาย: อัลกอริทึมใช้หน่วยความจำเพิ่มเติมสำหรับตัวแปรสเกลาร์เพียงไม่กี่ตัว ได้แก่ *total*, *maximum*, และ *average* ซึ่งใช้หน่วยความจำจำนวนคงที่ ไม่เพิ่มขึ้นตามขนาดของข้อมูลนำเข้า n

*หมายเหตุ: อาร์เรย์ *sales* ไม่นับเป็น Auxiliary Space เนื่องจากเป็น Input Data ที่มีอยู่แล้ว

ข้อ 4 Best Case และ Worst Case แตกต่างกันหรือไม่

คำตอบ: ไม่แตกต่างกัน (ทั้ง Best Case และ Worst Case เป็น $O(n)$)

- **Best Case (กรณีดีที่สุด):** ใช้เวลา $O(n)$ — ถึงแม้ข้อมูลจะเรียงลำดับเรียบร้อยแล้ว หรือค่ามากที่สุดจะอยู่ตำแหน่งแรก โปรแกรมก็ยังคงต้องวนลูปตรวจสอบข้อมูลทุกตัวในอาร์เรย์จนครบ n ตัว
- **Worst Case (กรณีแย่ที่สุด):** ใช้เวลา $O(n)$ — แม้ค่ามากที่สุดจะอยู่ตำแหน่งสุดท้าย หรือเรียงจากน้อยไปมาก โปรแกรมก็ต้องไล่ตรวจสอบจนครบ n ตัวเช่นเดียวกัน

สรุป: เนื่องจากฟังก์ชันนี้ใช้การวนลูปอ่านข้อมูลทุกตัวในอาร์เรย์โดยไม่มีเงื่อนไขหยุดการทำงานกลางคัน (Early Exit) จึงต้องตรวจสอบข้อมูลจนจบเสมอ ทำให้ Best Case และ Worst Case มีค่าเท่ากันคือ $O(n)$

ข้อ 5 หากแยกเขียนลูป 3 รอบ ความซับซ้อนจะเปลี่ยนจากวิธีลูปรอบเดียวหรือไม่

คำตอบ: ไม่เปลี่ยนแปลง ความซับซ้อนเชิงทฤษฎี (Big O) ยังคงเป็น $O(n)$ เท่าเดิม

การเปรียบเทียบ:

- **วิธีวนลูป 1 รอบ:** มีจำนวนการทำงานประมาณ n ครั้ง $\rightarrow O(n)$
- **วิธีแยกวนลูป 3 รอบ:** (ลูปที่ 1 หาผลรวม, ลูปที่ 2 หาค่าสูงสุด, ลูปที่ 3 หาค่าเฉลี่ยหรือประมวลผลเพิ่มเติม) มีจำนวนการทำงานประมาณ $3n$ ครั้ง $\rightarrow O(3n) = O(n)$

สาเหตุ: การวิเคราะห์ Big O สนใจเฉพาะอัตราการเติบโตของฟังก์ชันเมื่อ n มีขนาดใหญ่มากๆ จึงตัดค่าคงที่ (Constant Factor เช่น 3) ออก ทำให้ทั้งสองวิธีมี Big O เท่ากันคือ $O(n)$

ข้อ 6 แม้ Big O จะเท่ากัน เหตุใดวิธีลูปรอบเดียวจึงอาจทำงานเร็วกว่า

แม้ทั้งสองวิธีจะมี Big O เป็น $O(n)$ เท่ากัน แต่วิธีวนลูป 1 รอบ (Single Pass) ทำงานเร็วกว่าในทางปฏิบัติด้วยเหตุผลหลัก 3 ประการ:

1. **ลดจำนวนการเข้าถึงหน่วยความจำ (Memory Access & Cache Locality):** การวนลูป 1 รอบจะอ่านข้อมูลจาก RAM/Cache เพียงครั้งเดียว ขณะที่การวนลูป 3 รอบต้องเข้าถึงข้อมูลในอาร์เรย์เดิมซ้ำถึง 3 ครั้ง ทำให้ใช้ Memory Bandwidth มากกว่า
2. **ลด Loop Overhead:** การสร้างและจบลูปแต่ละรอบมีค่าใช้จ่ายประมวลผล (Instruction Overhead) เช่น การกำหนดค่าตัวนับ การเพิ่มค่าตัวนับ และการเช็คเงื่อนไขจบลูป การใช้ลูปเดียวจึงลดจำนวนคำสั่งควบคุมลูปรวมลง
3. **มี Constant Factor ที่ต่ำกว่า:** เมื่อพิจารณาเวลาทำงานจริงในรูป $T(n) = c \cdot n$ วิธีลูปรอบเดียวจะมีค่าคงที่ c น้อยกว่าวิธี 3 ลูป ส่งผลให้เวลาในการทำงานจริง (Execution Time) น้อยกว่า